

Multi-objective Model Rotations for Analog In-Memory Compute

1st William Isaac Trost
wit2102@columbia.edu

2nd David Zhiyuan Wang
dzw2109@columbia.edu

3rd Alvis Ou
ao2844@columbia.edu

Abstract—Analog in-memory compute (AIMC) has been shown to be a promising approach for improving the energy efficiency and throughput of large language model inference thanks to its ability to perform matrix operations directly inside memory arrays. It does come with a downside, however, and that is the fact that analog hardware can suffer from non-idealities, ranging from IR drop to ADC/DAC quantization, significantly degrading model quality. Mitigations such as hardware-aware training exist but these traditional methods are computationally expensive and difficult to apply to large pretrained language models.

In this work, we investigate whether the robustness to analog hardware effects in a lightweight post-training setting can be improved by orthogonal model rotations, inspired by SpinQuant. Using TinyLlama and IBM AIHWKIT, we implement both fixed and learned rotations under simulated analog inference conditions, and we evaluate random Hadamard rotations, online activation rotations, and learned rotations trained directly against analog language-modeling loss.

From our experiments, we have observed that the effects of rotations depend heavily on the target hardware non-ideality. Random Hadamard rotations, for one, improve quantization robustness but, at the same time, substantially worsen performance under large IR-drop conditions due to having increased energy spread across the crossbar. On the other hand, learned analog-aware rotations are able to partially recover perplexity relative to unrotated analog baselines by balancing outlier reduction and sparsity preservation. Our learning suggests that model rotations are a promising lightweight direction for improving AIMC robustness without requiring full hardware-aware retraining. <https://github.com/IsaacTrost/AnalogWeightRotations>

<https://wandb.ai/ao2844-columbia-university/aimc-rotations/reports/Final-report--VmldzoxNjgyNTE5Mw>

I. INTRODUCTION

Today, large language model inference is bottlenecked by data transfer. As model sizes and the amount of computation grows, the cost of transferring weights and activations between memory and compute becomes an increasingly significant portion of inference time. Analog in-memory compute (AIMC) addresses this bottleneck by performing matrix multiply and accumulate operations directly inside memristive crossbar memory arrays, enabling in-memory computation and more energy-efficient and higher throughput LLM inference [2].

Analog hardware works by replacing digital arithmetic with analog computation. In a typical resistive crossbar array, each weight is represented as a programmed conductance. Input activations are applied as voltages along the rows, and the resulting column currents accumulate according to Ohm’s law. This allows computation to occur with substantially less data movement than traditional digital hardware.

Despite the gains projected with AIMC, practical analog hardware suffers from physical non-idealities that distort the output of computation. Unlike digital errors, many analog errors are structured and input-dependent, meaning the amount of distortion depends on both the hardware layout and the distribution of activations and weights passing through the crossbar. These errors can accumulate across many transformer layers and significantly degrade language model quality.

We focus on two major non-idealities: IR drop and quantization. IR drop is the voltage loss along wires caused by their internal resistance. As signals propagate across the crossbar, voltages attenuate, making the effective computation depend on spatial location within the array. Quantization error is introduced by finite-precision DACs and ADCs when weights and activations are converted between discrete and continuous representations. Since activations and weights in LLMs often contain large outliers, these outliers dominate the available dynamic range and cause smaller values to lose precision.

Additional effects such as conductance noise and device programming error can further distort the signal and degrade model quality. These non-idealities are particularly challenging because different hardware effects depend on different properties of the weight and activation distributions, making mitigation difficult.

Traditional mitigation strategies, such as Hardware-Aware Training (HAT), retrain the model with simulated hardware errors in the loop is computationally expensive and requires substantial data and time to retrain. This motivates model-level techniques that mitigate analog error without significant time and cost.

Recent model rotation techniques like SpinQuant have successfully optimized models for digital quantization by transforming weight and activation matrices using a rotation matrix to reduce outliers. Our project explores whether orthogonal model rotations, inspired by SpinQuant, can be applied to mitigate the effects of analog non-idealities. Using TinyLlama and WikiText-2, we evaluate AIMC inference in AIHWKIT under IR-drop and quantization settings, comparing an unrotated baseline against random Hadamard rotations and trained rotations. Our results show that analog-aware trained rotations can substantially recover perplexity relative to unrotated baseline, suggesting that rotations are a promising lightweight approach for improving model robustness in AIMC.

II. LITERATURE REVIEW

A. Mitigating Analog Hardware Errors

A large body of prior work has focused on mitigating the impact of analog hardware non-idealities through training-time techniques. Hardware-aware training (HAT) incorporates analog effects such as noise, IR drop, and quantization directly into the forward pass during training, allowing the model to adapt its parameters accordingly. Prior work has shown that this approach can significantly improve robustness to hardware distortions [1], [2]. However, HAT is computationally expensive and requires access to training data, making it difficult to apply to large pretrained language models in practice.

As a result, there has been growing interest in post-training methods that improve robustness without retraining. Most of these approaches have been studied in the context of quantization, where the goal is to reduce the error introduced by mapping weights and activations to lower precision.

B. Post-Training Quantization Methods

Recent work on post-training quantization (PTQ) for large language models has focused on reshaping weight and activation distributions to better fit limited precision representations. For example, GPTQ [3] performs layer-wise second-order approximations to minimize quantization error, while SmoothQuant [4] rescales activations and weights to reduce the effect of activation outliers.

A common theme across these methods is that they target a single source of error (typically quantization) and optimize a corresponding proxy objective. While highly effective in digital settings, these techniques do not directly address other analog-specific effects such as IR drop or spatially varying distortions across crossbar arrays.

C. Rotation-Based Transformations

More recently, rotation-based methods have been proposed as a flexible post-training mechanism for improving model robustness. SpinQuant [5] introduces orthogonal rotations into transformer architectures to reduce activation outliers and improve quantization performance.

The key idea is that for an orthogonal matrix R , a linear transformation can be rewritten as

$$y = Wx = (WR^{-1})(Rx), \quad (1)$$

which allows the internal representation of the model to be transformed without changing its functional behavior. By choosing R appropriately, the distribution of activations and weights can be made more uniform, reducing sensitivity to quantization.

SpinQuant explores both structured (Hadamard-based) and learned rotations, showing that even fixed rotations can significantly reduce outliers, while learned rotations can further improve performance by directly optimizing a quantization-aware objective. In addition, the method introduces online rotations that are applied at runtime to intermediate activations, enabling further control over activation distributions.

One important aspect of this approach is that it operates entirely in a post-training setting and introduces minimal computational overhead, making it attractive for large-scale deployment.

D. Limitations of Existing Approaches

While rotation-based methods such as SpinQuant demonstrate strong improvements for quantization, they are still primarily designed for a single objective. In analog hardware, however, different non-idealities depend on different properties of the weight and activation distributions. For example, quantization error is driven by dynamic range and outliers, while IR drop depends on how signal energy is distributed spatially across crossbar columns. This suggests that optimizing for one objective may not fully address other sources of error, and may even introduce trade-offs.

E. Our Direction

Motivated by these observations, our work builds on rotation-based methods and explores their use in a broader analog setting. Rather than focusing on a single proxy objective, we investigate whether structured and learned rotations can improve robustness to multiple hardware effects, including quantization and IR drop, within a unified post-training framework.

Our approach remains lightweight and does not require full retraining, but instead adapts the idea of rotations to better align model representations with the behavior of analog hardware. This serves as the basis for the methods described in the next section.

III. METHODOLOGY

A. IBM aihwkit

AIHWKIT is an open source project from IBM designed to effectively simulate and test many pytorch layers and model types, with analog non-idealities injected in. It often does not perfectly simulate all circuit dynamics, as doing so would be prohibitively expensive for any crossbar of a reasonable size, but uses proxies that are differentiable (allowing for model training) and more efficient. Despite this, there is a substantial overhead in computation time and especially memory to working with aihwkit layers instead of raw pytorch layers. There are also many knobs that you can turn on the exact setup of the layers. We tested many configurations, and were especially interested in how rotations trained on one ported over to others, but the primary two effects were IR_drop and input/output quantization. We also simulated as if each layer of the model was defined on a single massive in memory compute tile. This is slightly unrealistic for the current state of the technology, but allowed simulations to work within memory constraint, and still produced useful results (each tile reserves several hundred megabytes of additional memory, so if you divide single layers into many tiles, usage quickly grows past reasonable limits.)

We settled on primarily training with IR drop value of 20, and 8 bit quantization. This is lighter quantization than what

spinquant was trying to train on (4 bit,) and we would expect it to be less damaging. This was chosen because ADCs and DACs are often 8 bit or 10 bit. We used an IR drop value of 20 for training. This is a high, but still realistic IR drop value. We picked these values with the aid of our mentors.

B. SpinQuant-Style Rotations

We implemented a rotation pipeline based on the main architectural ideas of SpinQuant, but adapted the objective to analog in-memory compute. The core transformation is an orthogonal change of basis applied to the transformer residual stream. For a hidden-state rotation R_1 , linear layers that read from the residual stream are rewritten by rotating their input side, while layers that write back into the residual stream are rewritten by rotating their output side. In our LLaMA-style models, this means applying R_1 to the input side of the q , k , v , gate, and up projections, and applying the inverse rotation to the output side of the attention output and MLP down projections. Since R_1 is orthogonal, this transformation preserves the floating-point model function when all paired rotations are applied exactly.

We also implemented the SpinQuant-style per-layer attention rotation R_2 on the value/output path. Unlike R_1 , which acts on the full hidden dimension, R_2 acts blockwise on each attention head dimension. For each transformer layer, R_2 is applied to the output blocks of the value projection and to the corresponding input blocks of the attention output projection. This allows the representation inside the attention value path to be changed without altering the exact floating-point computation. For the tinylama model, the R_1 rotation matrix is 2048x2048, or 4,194,304, and each R_2 matrix is 64x64, or 4096 params. Since there are 22 attention heads, this leads to a total param count of 4,284,416, compared to a 1.1b param model, so we only train roughly .4%.

We evaluated both fixed and learned rotations. The fixed rotations included identity, random orthogonal rotations, normalized Hadamard rotations, and signed Hadamard rotations of the form HD , where H is a normalized Hadamard matrix and D is a random diagonal sign matrix. For dimensions that were not powers of two, we used block-diagonal Hadamard matrices with the largest power-of-two block size that evenly divided the dimension. These rotations are norm-preserving, so any change in analog behavior comes from the changed coordinate distribution seen by the simulated hardware rather than from explicit rescaling of the model.

In addition to the offline residual-stream rotations, we tested online Hadamard rotations at two locations that are not fully controlled by R_1 and R_2 : the input to the attention output projection and the input to the MLP down projection. These correspond to the R_3 and R_4 rotations used in SpinQuant. In our implementation, the activation is rotated at runtime and the matching weight-side transformation is applied so that the floating-point computation remains unchanged. These online rotations were included because the attention output and MLP intermediate activations can contain large outliers that are especially important for low-bit quantization. We found these

were extremely damaging, however, when IR drop was used, as rotation matrices that successfully fight that are generally closer to the identity matrix, as destroying sparsity too much is damaging.

C. Rotation Training Objectives

After implementing the fixed rotations, we trained rotations directly against analog objectives. In the most successful regime, all target transformer projections were replaced with runtime-rotated analog linear layers. At each forward pass, the current R_1 and R_2 matrices were used to construct the effective rotated weights, which were then loaded into AIHWKIT analog layers. The model was trained using the language-modeling loss, while all original model weights remained frozen. Only the rotation matrices were optimized. This was meant to test whether rotations could be learned as a lightweight post-training adaptation to the analog hardware model, rather than requiring full hardware-aware finetuning of all weights.

Because the rotations must remain orthogonal to preserve the original model computation, we optimized them using a Stiefel-manifold optimizer with a Cayley-style retraction. This kept the learned matrices close to valid rotations during training. Since the training loss was noisy, especially under analog simulation, checkpoints were selected using the best observed training loss rather than the final training step. The optimizer used was taken from the spinquant codebase, and verified to ensure it maintained orthogonality with the training regime.

D. Evaluation Protocol

We evaluated rotations in two complementary ways. First, we used one-layer experiments to measure the direct analog output error of a single projection under different rotations and hardware presets. These tests compared the analog output against the exact floating-point output for the same layer, allowing us to study how rotations changed sensitivity to IR drop, weight noise, and quantization in isolation.

Second, we evaluated full-model perplexity on WikiText. For full-model evaluation, trained or fixed rotations were baked into the model weights, the relevant linear layers were converted to AIHWKIT analog layers, and perplexity was measured under the selected hardware configuration. We compared a floating-point prepared model, an analog identity baseline, and analog rotated models. This separation was important because a rotation can reduce single-layer analog error while still harming full-model perplexity if the hardware objective, online rotations, or static weight baking do not match the training setup exactly.

IV. RESULTS

A. One layer tests

For an isolated-layer study, we evaluated the layer 0 query projection of TinyLlama. We captured real input activations to this projection using forward hooks, extracted the corresponding weight matrix, and then tested several orthogonal

input-basis transformations. For each rotation R , we applied the paired transformation $x' = xR^T$ and $W' = WR^T$, which preserves the exact floating-point output while changing the values seen by the analog crossbar. We then loaded W' into an AIHWKIT analog linear layer and compared the analog output against the ideal floating-point output. This allowed us to measure how different rotations affect analog error for a single representative transformer projection under IR drop, quantization, weight noise, and PCM-style hardware settings. The results, showing the error between ideal and rotated output, as well as a grid heatmap showing the percent improvement or degradation, are in figure 1

gle-Layer Rotation Results — TinyLlama layers[0].self_attn.q_proj 2048→2048 (early attent

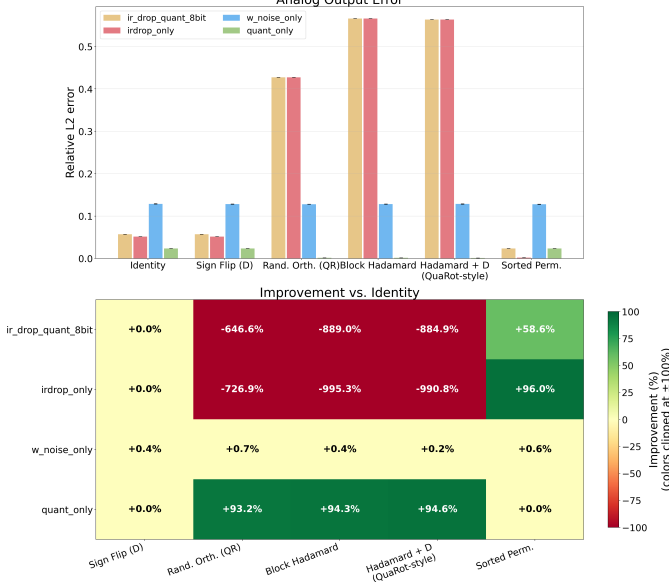


Fig. 1. Single layer tests under various rotation matrices

This reveals something very interesting, that the objectives to counter IR loss, and to counter quantization, are in opposition. Matrices like the random hadamard rotation matrix or even a random orthogonal matrix serve to distribute energy widely, and lesson outliers that are commonly found in layers such as this. This makes quantization more effective. This absolutely kills the models when it comes to high IR drop values though. The normal model layer does not get much worse with aihwkit ir drop, with ir drop only actually being the least damaging config (none of these will show any error under ideal configurations, by design of the rotations). This is because llama models, and most llms, are fairly sparse by design in both the activations and weights, meaning there are relatively few activations with non 0 output. Random matrices destroys this property, spreading energy and thus making most all of the activations and such non 0, giving more lanes for IR drop to seep in and cause issues perturbing the outputs.

It was this realization that led us to largely abandon the online random hadamard input rotations R_3 and R_4 . They add computational overhead to inference compared to running

the model cleanly, and the spreading nature will likely have a similar effect here of making IR drop worse.

We also realized training for any matrix designed to help with IR drop should start from the identity matrix. The sorted permutation matrix simply reorders the input channels using by mean absolute activation, trying to place the more relevant channels closer to the parts of the matrix that are less effected by IR drop (closer to the simulated DACs and ADCs.) This unfortunately does not scale to many different layers being affected by the same matrix, or different formulations of IR drop. It is simpler and more adaptable to let the training process start at the identity rotation matrix and learn something with the specific IR drop setup than to try to find the optimal one with that set of constraints.

B. Full model tests

To train the full tinyllama 1.1 model, we implemented the training scheme defined in the methodology section, minimizing language modeling loss over the 83% training split of the wikitext-2 dataset.

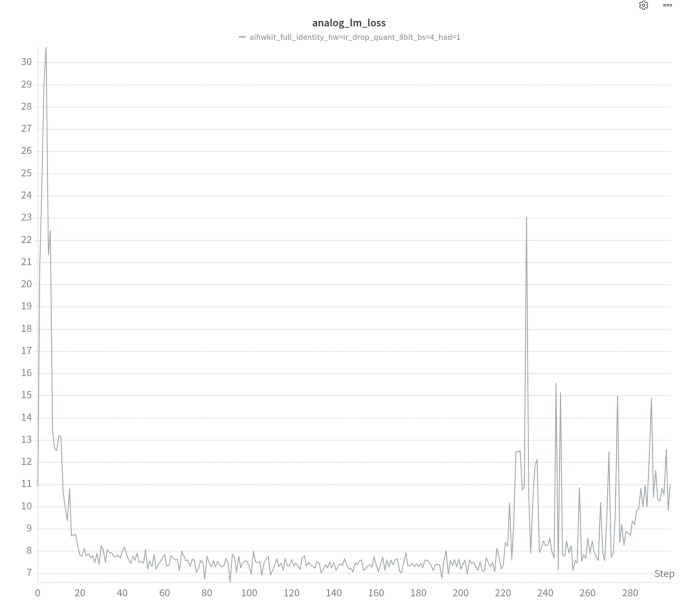


Fig. 2. Loss graph over run starting from identity and 8 bit quant

Figure 2 shows the loss graph over a training run, starting from the identity matrix. We can see that the loss is very noisy, and can fall into regimes or sections of the loss landscape that are very noisy and spiky. To account for this, we kept learning rate low, and selected the best checkpoint in terms of loss from the run.

Figure 3 shows the absolute weight values for several layers which are part of the attention head based on several different rotation matrices. The first is the base model, with no rotations applied, the second is a random hadamard rotation matrix, and the third is a trained weight matrix designed to counter high IR drop.

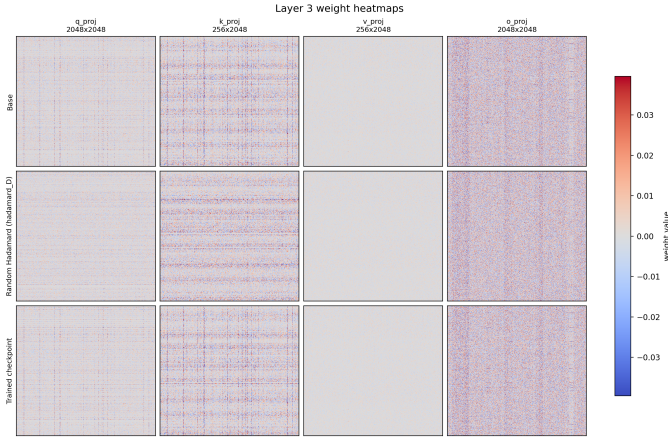


Fig. 3. Rotated weights of some layers, trained checkpoint is build to counter IR loss

Figure 3 shows something interesting. Weights designed to counter high IR drop are closer to the identity rotation matrix, having no effect on the weights of the model, than a random hadamard that is good for quantization.

Figure 4 further shows this, by examining the layer three inputs to the mlp section of the model

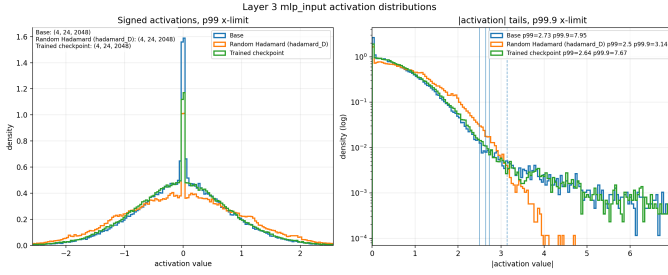


Fig. 4. Activation distribution over various rotation matrices

Specifically, this shows that the trained checkpoint smoothes out the distribution a bit, maintains much of the sparsity in activations, but slightly reduces the p99 and p99.9 of the activation absolute values. This corresponds to an improvement in the performance with quantization by reducing ranges, while also helping with IR drop.

Finally, we analyze how the checkpoint, trained against a rotation with 8 bit quantization and and IR drop value of 20, compares to no rotations at all (or the identity matrix) and r1 and r2 both initialized as random hadamard matrices

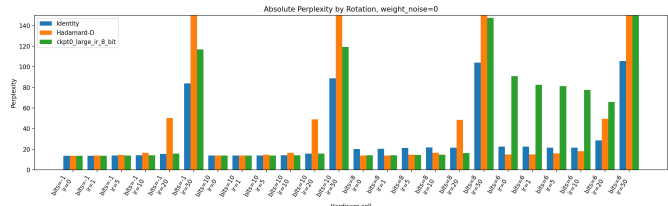


Fig. 5. Absolute values of the perplexity test with various rotation schemes, note hat several bars go of the scale

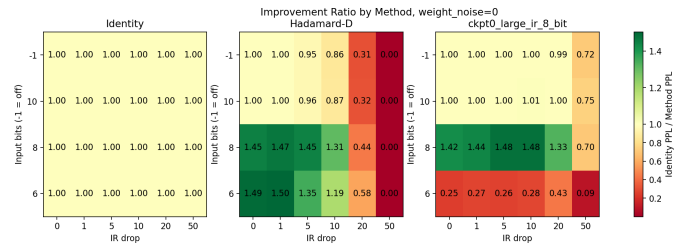


Fig. 6. Grid search of the rotation schemes

Figure 5 highlights several important trends. First, low-bit quantization noticeably degrades model performance, but large IR-drop values are even more damaging. Second, our learned rotation is effective in the hardware setting it was trained for, reaching a best-case perplexity of 17 compared with 23 for the identity rotation. This improvement also transfers to milder settings, including IR drop of 10 with 8-bit quantization, and remains competitive across most 8-bit quantization cases. Figure 6, which plots the ratio between the identity baseline and the random Hadamard or trained rotations, makes the trend clearer: the trained rotation improves performance near its training regime, but when pushed beyond that regime in either the IR-drop or quantization dimension, it often breaks down and can perform worse than doing nothing. We observed this pattern across many training settings. The random Hadamard rotation also behaves as expected from SpinQuant: its energy-spreading effect helps under quantization alone, but once substantial IR drop is introduced, its performance degrades sharply, as indicated would be the case in the 1 layer tests.

We also tested the online r3 and r4 matrices. We found that these were helpful under quantization only or light IR drop regimes, but similar to what we see here, the energy spreading effects obliterate model performance under substantial IR drop, and since they are online, there is no way to train them to be helpful (both would be too large to be trained offline, requiring more weights than the total model.) When we include the r3 and r4 matrices for a ir drop of 20, with no other rotation, we get a ppl of over 180000, compared to a ppl of 15.4 for doing nothing and just accepting the analog nonidealites, and a float ideal baseline of 13.6.

V. DISCUSSION

A. Interpretation

The experimental results demonstrate that weight and activation rotations mitigate the effects of IR drop and quantization and improve analog inference quality. But it also shows that optimal rotation depends strongly on the target hardware non-ideality. Random Hadamard rotations, as demonstrated in the experiment as well as in SpinQuant, are effective for reducing quantization error by spreading large weight and activation values across dimensions, reducing the number of outliers in the tensor. However, our results also show that the same energy spreading property may become harmful under significant IR drop. In our single-layer experiments, random Hadamard rotations shown improvements in the quant-only

environment against identity yet degraded the model in the IR-drop only environment. We attribute this observation to the fact that random orthogonal rotations reduce sparsity by distributing energy across more channels in the crossbar array. While this helps quantization, it creates more active current paths across the array, exacerbating the effects of IR drop across wires. As a result, random Hadamard rotations may be even harmful to model quality under significant IR drop. This tradeoff explains why learned rotations are important. A fixed rotation has no knowledge of the specific analog hardware configuration, while a trained rotation can adapt to the combined effects of quantization and IR drop, finding the best middle ground between sparsity and reducing outliers.

In the full-model experiments, the trained rotation was the most effective in analog environments, improving perplexity from 23 for the identity baseline to 17 in the best case. The learned rotation also transferred to some milder settings, such as lower IR drop with 8-bit quantization, but degraded when evaluated too far outside its training regime. This suggests that analog-aware rotation training learns a hardware-specific compromise rather than a universally optimal transformation.

The activation distribution graph further support this interpretation. Compared with a random Hadamard rotation, the trained rotation’s distribution remained closer to the identity transformation, preserving more of the original weight and activation structure. At the same time, it slightly smoothed the activation distribution and reduced high-percentile activation magnitudes shown by the reduction in p99 and p99.9 values. This indicates that the trained rotation does not simply maximize energy spreading like random Hadamard matrices. Instead, it aims to find the balance between preserving enough sparsity to limit IR-drop while reducing outliers enough to reduce quantization error.

B. Limitations

This project was limited by time and hardware resources. While we validated that fundamentally analog-aware rotation training can improve analog inference performance, we were not able to fully tune learning rates, checkpoint selection, initialization, or other training hyperparameters to test the limits of model rotations. Thus, the reported trained rotations should be viewed as initial evidence of the approach rather than fully optimized results.

Compute and memory limits also hindered the depth of the project. Training on a Nvidia T4 with 16GB of memory, we frequently encountered OOM errors in training and inferences, required simplifying the analog simulation. In particular, we used larger AIHWKIT tile mappings to keep full-model runs feasible, even though smaller (default 512 x 512) tiles would be more realistic for physical AIMC hardware. This may have exacerbated the effect of IR drop in our experiments as larger tiles are naturally more subject to effects of IR drop across wires. We also focused mainly on IR drop and ADC/DAC quantization, leaving more complex non-idealities such as conductance noise, drift, and programming error for future work.

C. Future Work

Future work should test analog-aware rotations across a broader range of AIMC settings, including different IR-drop strengths, quantization bit-widths, tile sizes, and additional hardware non-idealities. This would help determine how well a trained rotation generalizes beyond the configuration it was optimized for.

The next step is also to validate the method on real analog hardware, such as an IBM AIMC tile, since software simulation may not capture all physical effects. Finally, the approach should be evaluated on larger and more realistic LLMs to determine whether the observed perplexity recovery scales beyond the 1.1 billion parameter TinyLlama.

VI. CONCLUSION

In this project, we explored whether the robustness of large language models running on analog in-memory compute hardware can be improved by orthogonal model rotations. Building on ideas from SpinQuant, we were able to produce both fixed and learned rotations within an AIHWKIT-based analog simulation pipeline as well as evaluate their behavior under IR-drop and quantization conditions.

Our results show that different analog non-idealities place competing demands on the model representation. Random Hadamard rotations, representing rotations that distribute energy broadly across channels, are effective when it comes to reducing quantization error thanks to their ability to suppress activation outliers. The same spreading behavior, however, is significantly detrimental to IR-drop sensitivity as it increases the number of active signal paths across the crossbar. Through our experiment, we observed this tradeoff in both isolated-layer tests and full-model perplexity evaluation.

Simultaneously, we discovered that learned analog-aware rotations were able to partially balance these competing objectives. Compared to the analog identity baseline, trained rotations improved perplexity in the hardware regimes they were optimized for while also preserving more of the original sparsity structure than random Hadamard rotations. The learned rotations also generalized moderately to nearby hardware settings, though performance degraded outside the training regime.

Overall, our experiments suggest that model rotations are a promising lightweight post-training approach for improving AIMC robustness that requires no full hardware-aware re-training. Though limited by compute resources and simplified simulation assumptions, our work should have demonstrated that rotation-based adaptation can capture meaningful analog hardware behavior, providing a foundation for future work on multi-objective analog optimization.

ACKNOWLEDGMENT

The authors thank Charles Mackin and Akul Malhotra from IBM Research for their mentorship, technical guidance, and feedback throughout this project. We also thank the HPML course staff for their support during the semester.

REFERENCES

- [1] T. Gokmen and Y. Vlasov, "Acceleration of deep neural network training with resistive cross-point devices," *arXiv preprint arXiv: 1603.07341*, 2016.
- [2] M. Rasch, D. Moreda, T. Gokmen, et al., "A flexible and fast PyTorch toolkit for simulating training and inference on analog crossbar arrays," *arXiv preprint arXiv:2104.02184*, 2021.
- [3] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pretrained Transformers," *arXiv preprint arXiv:2210.17323*, 2022.
- [4] G. Xiao, J. Lin, M. Seznec, et al., "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," *arXiv preprint arXiv:2211.10438*, 2022.
- [5] Z. Liu, C. Zhao, I. Fedorov, et al., "SpinQuant: LLM Quantization with Learned Rotations," *arXiv preprint arXiv:2405.16406*, 2024.

VII. APPENDIX

A. AI use disclosure

****Tool(s) used:**** *GitHub Copilot, Cursor*

****Specific purpose:**** *Wrote boilerplate templates and plotting code, helped explore OOM errors (did not write code to fix errors), fixed documentation and wrote more expressive code comments when necessary, proofread report to check for errors and inconsistencies.*

****Sections affected:**** *e.g., src/plot code comment changes were in all src files*

****How we verified correctness:**** *All code was reviewed for correctness. Fully AI generated code was not used in critical sections (only plotting) and was read to confirm correctness. AI comments and proofreading suggestions were read and verified for correctness.*

By submitting this project, the team confirms that the analysis, interpretations, and conclusions are our own, and that any AI assistance is fully disclosed above. The same disclosure block appears as an appendix in the final report.